INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DISCIPLINA: Padrões de Projeto em Aplicações Web
PROFESSOR: Éder Pereira
ATIVIDADE PRÁTICA: Padrão Estrutural Composite

Contextualização do Problema

Estamos desenvolvendo o módulo de navegação de um sistema de mapas para calcular trajetos entre diferentes pontos. Um trajeto completo pode ser percorrido de diversas maneiras (a pé, de carro, de ônibus, de trem, etc.), e cada tipo de deslocamento possui sua própria forma de exibição das direções e distâncias.

No entanto, um trajeto maior (um `Caminho`) é na verdade composto por uma sequência de múltiplos trechos individuais, ou até mesmo por subcaminhos já agrupados. Para que o código cliente consiga tratar tanto um único trecho simples quanto uma rota inteira composta de forma uniforme e transparente, utilizaremos o padrão estrutural **Composite**.

O objetivo do **Composite** é agrupar objetos em estruturas de árvore para representar hierarquias parte-todo, permitindo que objetos individuais e composições de objetos sejam tratados de maneira idêntica por meio de uma interface comum (`Trecho`).

Roteiro de Implementação

1. **Criação do Projeto:**
    - Crie um projeto Java no Eclipse chamado `Composite`.
2. **Definição da Interface Componente (`Trecho`):**
    - Crie a interface `Trecho.java` com a assinatura do método necessário para a exibição das instruções.

```java
public interface Trecho {
    void imprime();
}
```

3. **Modelagem dos Folhas (`TrechoAndando` e `TrechoDeCarro`):**
    - Crie as classes concretas `TrechoAndando.java` e `TrechoDeCarro.java` que implementam a interface `Trecho`.
    - Defina os atributos necessários (como `direcao` e `distancia`), seus construtores e a assinatura do método `imprime()`.

```java
public class TrechoAndando implements Trecho {
    private String direcao;
    private double distancia;

    public TrechoAndando(String direcao, double distancia) {
        this.direcao = direcao;
        this.distancia = distancia;
    }

    // Comportamento: Exibir as instru  es informando que o trecho e a
       pe, mostrando a direcao e a distancia percorrida em metros.
    public void imprime() {
        // TODO: Implementar a impressao das instrucoes do trecho a pe
    }
```

```
}

public class TrechoDeCarro implements Trecho {
    private String direcao;
    private double distancia;

    public TrechoDeCarro(String direcao, double distancia) {
        this.direcao = direcao;
        this.distancia = distancia;
    }

    // Comportamento: Exibir as instru  es informando que o trecho e de
       carro, mostrando a direcao e a distancia percorrida em metros.
    public void imprime() {
        // TODO: Implementar a impressao das instrucoes do trecho de carro
    }
}
```

4. **Modelagem do Contêiner Composite (`Caminho`):**

   - Crie a classe `Caminho.java` que também implementa `Trecho`, permitindo que um caminho seja composto por uma coleção de objetos do tipo `Trecho` (folhas ou outros composites).
   - Defina uma lista interna para armazenar os trechos, além dos métodos de gerenciamento e da delegação da impressão.

```
import java.util.ArrayList;
import java.util.List;

public class Caminho implements Trecho {
    // TODO: Definir a cole  o para armazenar objetos do tipo Trecho

    public Caminho() {
        // TODO: Inicializar a lista de trechos
    }

    // Comportamento: Adicionar um novo Trecho (simples ou composto) ao
       caminho.
    public void adiciona(Trecho trecho) {
        // TODO: Implementar a inclus o do trecho
    }

    // Comportamento: Remover um Trecho do caminho.
    public void remove(Trecho trecho) {
        // TODO: Implementar a remocao do trecho
    }

    // Comportamento: Percorrer a cole  o de trechos executando o metodo
        imprime() de cada elemento.
    public void imprime() {
        // TODO: Iterar sobre os trechos e invocar o imprime() de forma
           polimorfica
    }
}
```

5. **Testes do Sistema (`TestaCaminho`):**

   - Crie a classe de testes montando trajetos simples e encadeando caminhos dentro de outros caminhos para validar a estrutura em árvore.

```java
public class TestaCaminho {
    public static void main(String[] args) {
        // TODO: Criar instancias de TrechoAndando e TrechoDeCarro com mensagens e distancias de teste
        // TODO: Instanciar um objeto Caminho (caminho1) e adicionar trechos a ele
        // TODO: Imprimir o conteudo do caminho1
        // TODO: Instanciar um segundo Caminho (caminho2) e adicionar a ele o caminho1 e outros trechos
        // TODO: Imprimir o conteudo do caminho2 demonstrando o tratamento uniforme
    }
}
```